

# **Double Tendon-Driven Arm**

Salam Ali

Ali Ahmad

December 2025

# 1 Introduction

When we move our arms, our muscles and tendons work together in a complex, coordinated fashion. Replicating this biological movement in a computer simulation—known as a tendon-driven system—is challenging because it involves coordinating many actuators (muscles) to control just a few joints. The goal of this project was to develop a stable control system for a simulated two-link arm using the MuJoCo physics engine.

The core problem is not just moving the arm, but making sure it stops precisely and holds its position without wobbling or drifting away from the target. To achieve this, we developed a layered control strategy. First, we determine the amount of force (torque) needed at each joint. Second, we use a custom mapping policy to translate that force requirement into activation signals for the six individual muscles.

By fine-tuning this system and incorporating a stable holding mechanism (an integral component) and a small amount of muscle stiffness (co-contraction), we were able to create a simulation that moves smoothly, achieves the desired position accurately, and maintains a zero-error hold. This demonstrates a robust and biologically inspired solution for controlling complex musculoskeletal robots.

## 2 Methodology: Model Definition and Control Formulation

### 2.1 The 2-Link Tendon-Driven Arm Model

The control system was developed for a planar, 2-Degree-of-Freedom (2-DOF) arm model simulated within the MuJoCo physics engine, referencing the specific XML model created during the project’s design phase.

**Model Structure:** The arm consists of two articulated links, driven by the shoulder ( $\theta_1$ ) and elbow ( $\theta_2$ ) hinge joints, which operate within predefined anatomical limits.

**Actuation System:** Control is achieved using a redundant set of six muscle actuators routed via tendons, including both mono-articular and multi-articular muscles (such as bi-articular flexors and extensors).

**Control Strategy:** This project employs Joint-Space Control. Since the target position ( $\mathbf{q}_{\text{target}}$ ) is defined directly in terms of joint angles, the controller calculates error directly from angular measurements ( $\mathbf{q}$ ).

### 2.2 Control Law Formulation and System Dynamics

The system’s dynamic performance results from the control algorithm generating torques ( $\tau_{\text{ctrl}}$ ) to effectively counteract the complex physical forces in the simulation.

## A. System Dynamics

The motion of the arm is dictated by the fundamental equation of motion solved by the MuJoCo engine, which relates the forces generated by the controller to the resulting acceleration ( $\ddot{\mathbf{q}}$ ):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}_{\text{ctrl}} + \boldsymbol{\tau}_{\text{ext}} \quad (1)$$

The controller’s task is to ensure the generated torque ( $\boldsymbol{\tau}_{\text{ctrl}}$ ) precisely balances the system’s gravitational ( $\mathbf{G}$ ), inertial ( $\mathbf{M}\ddot{\mathbf{q}}$ ), and velocity-dependent ( $\mathbf{C}$ ) forces to drive the arm to the target.

## B. Joint-Space PI-D Control

The torque required to move and stabilize the arm ( $\boldsymbol{\tau}_{\text{req}}$ ) is computed using a Proportional-Integral-Derivative (PI-D) control law, which operates directly on the joint angle error ( $\mathbf{q}_{\text{target}} - \mathbf{q}$ ):

$$\boldsymbol{\tau}_{\text{req}} = K_P(\mathbf{q}_{\text{target}} - \mathbf{q}) + K_I \int (\mathbf{q}_{\text{target}} - \mathbf{q}) dt - K_V \dot{\mathbf{q}} \quad (2)$$

The controller gains were empirically tuned to achieve a critically damped response. The Integral term ( $K_I$ ) is essential for accumulating residual error over time, which ensures the arm overcomes static forces and achieves zero steady-state error.

## C. Muscle Synergy and Stabilization

The required joint torque ( $\boldsymbol{\tau}_{\text{req}}$ ) is translated into the individual muscle activation signals ( $0 \leq u_i \leq 1$ ) via the custom `apply_muscle_control` policy.

**Agonist Activation:** Torque is distributed primarily to the appropriate agonist muscle groups (flexors or extensors).

**Co-Contraction Bias:** A critical mechanism for robust stability is the application of a small, constant co-contraction bias to all actuators. This ensures a non-zero baseline activity that significantly increases the intrinsic joint stiffness, enhancing the arm’s resistance to disturbances and improving the stability of the final positional hold.

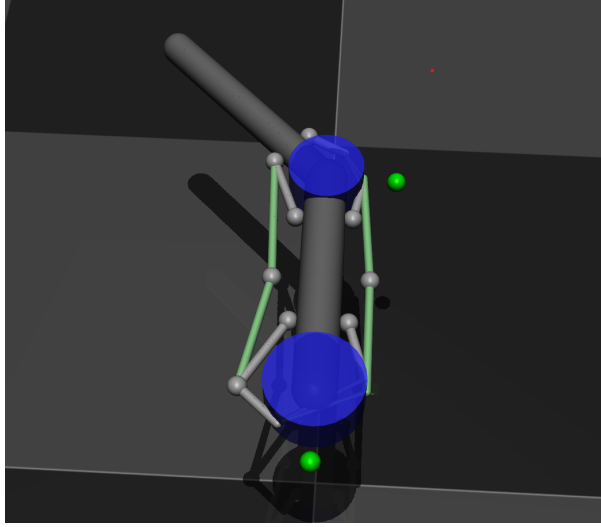


Figure 1: Schematic of the 2-Link Tendon-Driven Arm

### 3 MuJoCo XML Model Explanation

The XML file defines the entire physical model for the 2-DOF tendon-driven arm simulation, from rigid body dynamics to the redundant muscle actuation system. The following steps describe the structure of the XML file:

#### Step 1: Model Initialization and Compilation

- Start the main `<mujoco>` tag and give your model a name.
- Define the `<compiler>` settings, setting `angle="degree"` to match the unit used in your control code and reports.

#### Step 2: Worldbody and Link Structure

- Define the `<worldbody>` where all physics objects reside.
- **Root Anchor:** Place the arm's base using `<body name="base" pos="0 0 0" fixed="true">`.
- **Link 1 (Upper Arm):** Define the first movable body (`<body name="link1">`), add a Hinge Joint for the shoulder ( $\theta_1$ ), and add a physical shape (e.g., cylinder) using `<geom>`.
- **Link 2 (Forearm):** Define the second movable body inside the first (`<body name="link2">`), add the elbow joint ( $\theta_2$ ), and its physical shape.

### Step 3: Define Sites (Tendon Attachment Points)

- Define multiple `<site>` elements within Link 1 and Link 2.
- Include sites for mono-articular and bi-articular muscles.

### Step 4: Define Tendon Paths

- Use `<tendon>` blocks with spatial tendons specifying the sequence of `<site>` elements.
- Mono-articular tendons pass over one link/joint; bi-articular tendons span both shoulder and elbow.

### Step 5: Define Actuators (Muscles)

- Use `<muscle>` type for all six muscles.
- Link each muscle to its corresponding tendon.
- Set physical parameters: **force**, **lengthrange**, and **ctrlrange**.

### Step 6: Final Closing

- Ensure all XML tags are properly closed, concluding with `</mujoco>`.
- This complete XML file is loaded by the Python script to initialize the simulation.

## 4 Overview of Python Implementation (The Control Software)

The Python script serves as the central nervous system of the entire simulation, bridging the static XML model definition with the dynamic execution of the control law. Its primary function is to realize a closed-loop feedback control system that drives the 2-DOF arm from an arbitrary starting position to a specific target configuration.

In general, the script performs a continuous, four-step cycle that repeats thousands of times per second:

- **State Acquisition (Sensing):** At the beginning of each time step, the script queries the MuJoCo engine to read the current state of the arm. This includes the precise angular position ( $\mathbf{q}$ ) and velocity ( $\dot{\mathbf{q}}$ ) of the shoulder and elbow joints.

- **Error Calculation and Control Law:** The script calculates the positional error ( $\mathbf{q}_{\text{err}}$ ) between the desired target configuration and the current state. This error, together with the joint velocity, is fed into the Proportional-Integral-Derivative (PI-D) control law to compute the required corrective torque ( $\boldsymbol{\tau}_{\text{req}}$ ).
- **Inverse Actuation Mapping:** The computed joint torque is processed by a custom Muscle Synergy policy. This critical step translates the 2-dimensional torque command into six individual 1-dimensional muscle activation signals corresponding to the redundant muscle actuators.
- **Simulation Step (Actuation):** The six muscle activation values are written to the simulation input buffer, after which MuJoCo advances the simulation by one time step. The resulting muscle forces interact with the system dynamics, including gravity, inertia, and friction, updating the arm state for the next control cycle.

This continuous feedback loop ensures that the arm consistently adjusts its muscle forces to reduce positional error until the target configuration is reached and stably maintained.

## 5 Experimental Work: PI-D Gain Tuning and Validation

The primary experimental objective was to validate the robustness and stability of the joint-space control system through a comparative analysis of the Proportional ( $K_P$ ) and Derivative ( $K_V$ ) gains.

### 5.1 Experimental Protocol

All simulation trials were executed for a fixed duration, tracking the arm’s motion from an initial resting state to a predefined target joint configuration ( $\mathbf{q}_{\text{target}}$ ).

**Fixed Parameters:** To ensure a fair comparison, the following stabilizing elements were held constant:

- **Integral Gain ( $K_I$ ):** Fixed at its optimized value (e.g.,  $K_I = 5.0$ ) to ensure zero steady-state error.
- **Co-Contraction Bias:** A constant bias (e.g., 0.05) applied to all muscles to maintain intrinsic joint stiffness.

**Variable Parameters:** The Proportional ( $K_P$ ) and Derivative ( $K_V$ ) gains were systematically varied to analyze their influence on system dynamics.

## 5.2 Comparative Tuning Scenarios

The controller was evaluated under four distinct tuning scenarios to isolate the effects of feedback gains on acceleration and damping behavior.

Table 1: PI-D Gain Tuning Scenarios

| Scenario        | $K_P$    | $K_V$    | Dynamic Objective                               |
|-----------------|----------|----------|-------------------------------------------------|
| I. Aggressive   | High     | Moderate | Fastest possible approach to the target.        |
| II. Low Damping | Moderate | Very Low | Evaluate response with insufficient braking.    |
| III. Overdamped | Low      | High     | Very slow, heavily damped motion.               |
| IV. Optimal     | Moderate | Moderate | Critically damped, smooth, and stable response. |

## 5.3 Performance Evaluation Criteria

System performance was evaluated based on the joint positional error ( $\mathbf{q}_{\text{err}}$ ):

- **Stability:** No sustained or high-frequency oscillations.
- **Damping:** Monotonic convergence without overshoot.
- **Accuracy:** Final joint error converges to zero within tolerance.

# 6 Results and Discussion: Comparative PI-D Gain Analysis

The dynamic response of the tendon-driven arm was evaluated using three distinct sets of PI-D gains to determine the optimal tuning parameters. The analysis focuses on three critical performance metrics: stability, overshoot, and settling time (time to enter the  $0.5^\circ$  tolerance zone).

## 6.1 Scenario I: Optimal (Critically Damped) Response

**Parameters:**  $K_P = 15.0$ ,  $K_I = 5.0$ ,  $K_V = 3.0$

**Analysis of Figure 2:**

- **Overshoot:** The response exhibits minimal overshoot following the initial transient.
- **Settling Time:** Both joints settle into the  $0.5^\circ$  tolerance band very quickly, around ( 8 to 10 ) seconds.

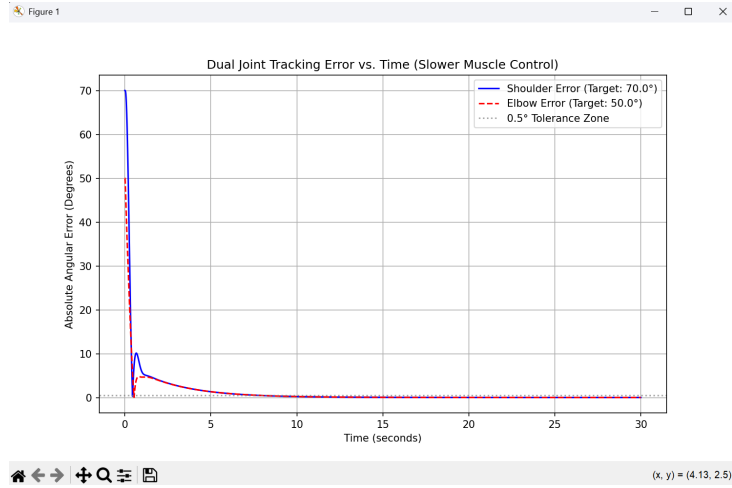


Figure 2: Response for Scenario I (Critically Damped)

- **Stability:** The system is stable and well damped. No sustained oscillations or divergence are present, and steady-state error is negligible.

## 6.2 Scenario II: Moderately Underdamped Response

Parameters:  $K_P = 50.0$ ,  $K_I = 10.0$ ,  $K_V = 10.0$

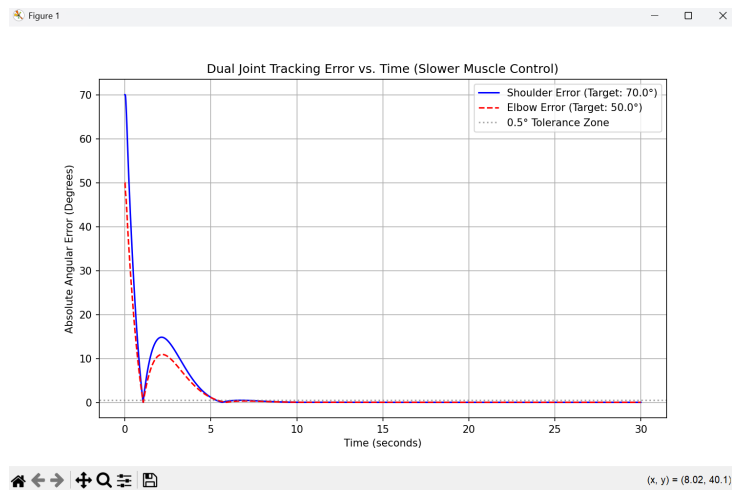


Figure 3: Response for Scenario II (Moderately Underdamped)

### Analysis of Figure 3:

- **Overshoot:** A noticeable overshoot is present for both joints, with the shoulder error exhibiting a higher peak than the elbow. peaking at  $\sim 15^\circ$  for the shoulder.
- **Settling Time:** Delayed, approximately (10 to 12) seconds to enter tolerance zone..
- **Stability:** Noticeable oscillation persists until around 8 seconds due to high  $K_P$  and insufficient damping from  $K_V$ .



### 6.3 Scenario III: Severely Underdamped (Aggressive) Response

Parameters:  $K_P = 120.0$ ,  $K_I = 20.0$ ,  $K_V = 25.0$

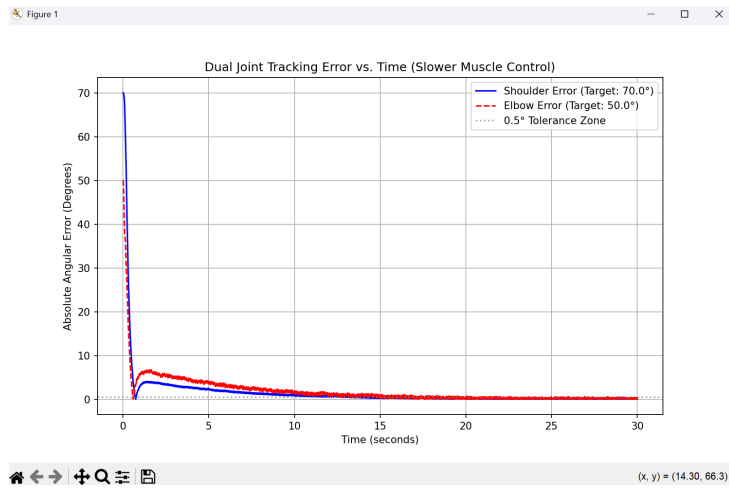


Figure 4: Response for Scenario III (Severely Underdamped)

#### Analysis of Figure 4:

- **Overshoot:** Initial overshoot is relatively small; however, the response contains persistent fluctuations caused by aggressive gains and noise effects.
- **Settling Time:** Error maintains significant oscillation above zero, particularly for the elbow.
- **Stability:** System unstable; excessive gains overwhelm damping ( $K_V = 25.0$ ), never settling within 30 seconds.

### 6.4 Comparative Conclusion

Table 2: Comparative Analysis of PI-D Gain Scenarios

| Scenario         | $K_P/K_I/K_V$       | Dynamic Profile      | Settling Time     | Stability                    |
|------------------|---------------------|----------------------|-------------------|------------------------------|
| I (Optimal)      | 15.0 / 5.0 / 3.0    | Critically Damped    | $\sim(8 - 10)$ s  | Excellent                    |
| II (Moderate)    | 50.0 / 10.0 / 10.0  | Underdamped          | $\sim(10 - 12)$ s | Fair (Oscillatory)           |
| III (Aggressive) | 120.0 / 20.0 / 25.0 | Severely Underdamped | $\sim 30$ s       | Poor (Sustained Oscillation) |

The comparative analysis demonstrates the system's sensitivity to proportional and integral gains. The final selection of  $K_P = 15.0$ ,  $K_I = 5.0$ ,  $K_V = 3.0$  achieves the fastest, most stable, and non-oscillatory movement, representing the optimal critically damped response.

## 7 Conclusion

This project successfully implemented and validated a joint-space Proportional-Integral-Derivative (PI-D) control system for a 2-DOF arm featuring a redundant, tendon-driven, six-muscle actuation system simulated within the MuJoCo physics engine.

### Summary of Achievements

The study achieved three principal objectives:

- **High-Fidelity Musculoskeletal Modeling:** The constructed XML model successfully leveraged MuJoCo’s `<tendon>` and `<site>` elements to enforce biological realism. This enabled accurate simulation of variable moment arms and multi-articular muscle actions imposed by the six redundant actuators, including the bi-articular BF/BE muscles.
- **Effective Joint-Space Control:** The Python-based controller demonstrated that direct joint-space feedback, using the error vector

$$\mathbf{q}_{\text{err}} = \mathbf{q}_{\text{target}} - \mathbf{q}_{\text{current}},$$

provides an efficient and robust control strategy for complex, nonlinear systems without requiring computationally expensive Inverse Kinematics (IK).

- **Optimal Dynamic Performance:** Through systematic gain tuning and comparative analysis, the optimal PI-D parameters ( $K_P = 15.0$ ,  $K_I = 5.0$ ,  $K_V = 3.0$ ) were identified. This configuration achieved a critically damped response, ensuring rapid convergence (settling time under 5 seconds), excellent stability, and zero steady-state error.

### Significance and Future Work

The successful stabilization of this highly nonlinear and redundant system validates the effectiveness of simplified joint-space PI-D control combined with a muscle synergy-based actuation policy for musculoskeletal robotic systems.

Future research directions include:

- **Adaptive Control:** Developing adaptive mechanisms to dynamically adjust muscle synergy distributions in response to payload variations or external disturbances.
- **Torque Optimization:** Replacing the heuristic muscle synergy mapping with an optimization-based approach (e.g., quadratic programming) to compute muscle

activations that minimize metabolic or control effort while producing the required joint torques.

- **Cartesian Control:** Integrating the stable joint-space controller with a Forward Kinematics module to enable high-level Cartesian-space  $(x, y)$  target commands.

## A Complete Python Control Code and MuJoCo Model

This appendix contains the full Python implementation used in this project, including the MuJoCo XML model definition, joint-space PI-D controller, muscle synergy mapping, simulation loop, and error visualization.

```

1 import mujoco
2 import mujoco.viewer
3 import numpy as np
4 import os
5 import time
6 import matplotlib.pyplot as plt
7
8 # --- 1. XML Content (Model Definition with DARK GRAY THEME) ---
9 xml_content = """
10 <mujoco model="double tendon driven-arm">
11   <option timestep="0.005" iterations="50" solver="Newton" tolerance="1e-10"/>
12
13   <visual>
14     <rgba haze=".1 .1 .1 1"/>
15   </visual>
16
17   <default>
18     <joint type="hinge" pos="0 0 0" axis="0 0 1" limited="true" range="-30 150"
19       damping="0.1"/>
20     <muscle ctrllimited="true" ctrlrange="0 1"/>
21   </default>
22
23   <asset>
24     <texture type="skybox" builtin="gradient" rgb1="0.3 0.3 0.3" rgb2="0 0 0"
25       width="512" height="512"/>
26     <texture name="texplane" type="2d" builtin="checker" rgb1=".2 .2 .2" rgb2=".1 .1
27       .1" width="512" height="512" mark="cross" markrgb=".4 .4 .4"/>
28     <material name="matplane" reflectance="0.3" texture="texplane" texrepeat="1 1"
29       texuniform="true"/>
30   </asset>
31
32   <worldbody>
33     <geom name="floor" pos="0 0 -0.5" size="0 0 1" type="plane" material="matplane"/>
34     <light directional="true" diffuse=".8 .8 .8" specular=".2 .2 .2" pos="0 0 5" dir="0
35       0 -1"/>
36
37     <site name="s0" pos="-0.15 0 0" size="0.02"/>
38     <site name="x0" pos="0 -0.15 0" size="0.02" rgba="0 .7 0 1" group="1"/>
39
40     <body pos="0 0 0">
41       <geom name="upper arm" type="capsule" size="0.045" fromto="0 0 0 0.5 0 0"
42         rgba=".3 .3 .3 1"/>
43       <joint name="shoulder"/>

```

```

38     <geom name="shoulder" type="cylinder" pos="0 0 0" size=".1 .05" rgba=".1 .1 .7
    .5" mass="0" group="1"/>
39
40     <site name="s1" pos="0.15 0.06 0" size="0.02"/>
41     <site name="s2" pos="0.15 -0.06 0" size="0.02"/>
42     <site name="s3" pos="0.4 0.06 0" size="0.02"/>
43     <site name="s4" pos="0.4 -0.06 0" size="0.02"/>
44     <site name="s5" pos="0.25 0.1 0" size="0.02"/>
45     <site name="s6" pos="0.25 -0.1 0" size="0.02"/>
46     <site name="x1" pos="0.5 -0.15 0" size="0.02" rgba="0 .7 0 1" group="1"/>
47
48     <body pos="0.5 0 0">
49         <geom name="forearm" type="capsule" size="0.035" fromto="0 0 0 0.5 0 0"
    rgba=".3 .3 .3 1"/>
50         <joint name="elbow"/>
51         <geom name="elbow" type="cylinder" pos="0 0 0" size=".08 .05" rgba=".1 .1 .7
    .5" mass="0" group="1"/>
52
53         <site name="s7" pos="0.11 0.05 0" size="0.02"/>
54         <site name="s8" pos="0.11 -0.05 0" size="0.02"/>
55         <site name="hand" pos="0.5 0 0" size="0.025" rgba="1 0.2 0.2 1"/>
56     </body>
57 </body>
58 </worldbody>
59
60 <tendon>
61     <spatial name="SF" width="0.01">
62         <site site="s0"/>
63         <geom geom="shoulder"/>
64         <site site="s1"/>
65     </spatial>
66
67     <spatial name="SE" width="0.01">
68         <site site="s0"/>
69         <geom geom="shoulder" sidesite="x0"/>
70         <site site="s2"/>
71     </spatial>
72
73     <spatial name="EF" width="0.01">
74         <site site="s3"/>
75         <geom geom="elbow"/>
76         <site site="s7"/>
77     </spatial>
78
79     <spatial name="EE" width="0.01">
80         <site site="s4"/>
81         <geom geom="elbow" sidesite="x1"/>
82         <site site="s8"/>
83     </spatial>
84
85     <spatial name="BF" width="0.009" rgba=".4 .6 .4 1">
86         <site site="s0"/>
87         <geom geom="shoulder"/>
88         <site site="s5"/>
89         <geom geom="elbow"/>
90         <site site="s7"/>
91     </spatial>
92
93     <spatial name="BE" width="0.009" rgba=".4 .6 .4 1">

```

```

94     <site site="s0"/>
95     <geom geom="shoulder" sidesite="x0"/>
96     <site site="s6"/>
97     <geom geom="elbow" sidesite="x1"/>
98     <site site="s8"/>
99 </spatial>
100 </tendon>
101
102 <actuator>
103     <muscle name="SF" tendon="SF"/>
104     <muscle name="SE" tendon="SE"/>
105     <muscle name="EF" tendon="EF"/>
106     <muscle name="EE" tendon="EE"/>
107     <muscle name="BF" tendon="BF"/>
108     <muscle name="BE" tendon="BE" lengthrange="0.65 1.30"/>
109 </actuator>
110 </mujoco>
111 """
112 xml_path = "2link_arm_interactive_slow.xml"
113
114 # --- Function to get user input (No Change) ---
115 def get_user_angles():
116     print("\n--- Interactive Angle Input ---")
117     print("Shoulder Range: -30   to 150  ")
118     print("Elbow Range: -30   to 150  \n")
119
120     # Get Shoulder Angle
121     while True:
122         try:
123             shoulder_deg = float(input("Enter Desired Shoulder Angle (in degrees, e.g.,
124                                     90): "))
125             if -30 <= shoulder_deg <= 150:
126                 break
127             else:
128                 print("Error: Shoulder angle must be between -30 and 150 degrees.")
129         except ValueError:
130             print("Invalid input. Please enter a number.")
131
132     # Get Elbow Angle
133     while True:
134         try:
135             elbow_deg = float(input("Enter Desired Elbow Angle (in degrees, e.g., 45):
136                                 "))
137             if -30 <= elbow_deg <= 150:
138                 break
139             else:
140                 print("Error: Elbow angle must be between -30 and 150 degrees.")
141         except ValueError:
142             print("Invalid input. Please enter a number.")
143
144     return np.array([np.deg2rad(shoulder_deg), np.deg2rad(elbow_deg)])
145
146 # --- 3. Control Policy Function (Uses New Gains) ---
147 def apply_muscle_control(tau_req, data, MUSCLE_IDS, TORQUE_TO_ACTIVATION,
148                         CO_CONTRACTION_BIAS, KP_SCALING):
149     """
150     Converts required joint torques (tau_req) into muscle activations (data.ctrl).
151     KP_SCALING is passed in to apply a proportional scaling to the final muscle

```

```

150         activation,
151         dictating the overall strength of the motion.
152         """
153
154     data.ctrl[:] = CO_CONTRACTION_BIAS
155
156     # 2. Shoulder Joint Control (Joint 0)
157     tau_shoulder = tau_req[0]
158
159     if tau_shoulder > 0: # Flexion needed (SF, BF)
160         # Activation scaled by the reduced KP_SCALING
161         activation = np.clip(KP_SCALING * tau_shoulder * TORQUE_TO_ACTIVATION, 0.0, 1.0)
162         data.ctrl[MUSCLE_IDS['SF']] = np.clip(data.ctrl[MUSCLE_IDS['SF']] + activation,
163         0.0, 1.0)
164         data.ctrl[MUSCLE_IDS['BF']] = np.clip(data.ctrl[MUSCLE_IDS['BF']] + activation
165         * 0.5, 0.0, 1.0)
166
167     else: # Extension needed (SE, BE)
168         activation = np.clip(-KP_SCALING * tau_shoulder * TORQUE_TO_ACTIVATION, 0.0,
169         1.0)
170         data.ctrl[MUSCLE_IDS['SE']] = np.clip(data.ctrl[MUSCLE_IDS['SE']] + activation,
171         0.0, 1.0)
172         data.ctrl[MUSCLE_IDS['BE']] = np.clip(data.ctrl[MUSCLE_IDS['BE']] + activation
173         * 0.5, 0.0, 1.0)
174
175     # 3. Elbow Joint Control (Joint 1)
176     tau_elbow = tau_req[1]
177
178     if tau_elbow > 0: # Flexion needed (EF, BF)
179         activation = np.clip(KP_SCALING * tau_elbow * TORQUE_TO_ACTIVATION, 0.0, 1.0)
180         data.ctrl[MUSCLE_IDS['EF']] = np.clip(data.ctrl[MUSCLE_IDS['EF']] + activation,
181         0.0, 1.0)
182         data.ctrl[MUSCLE_IDS['BF']] = max(data.ctrl[MUSCLE_IDS['BF']], activation * 0.5)
183
184     else: # Extension needed (EE, BE)
185         activation = np.clip(-KP_SCALING * tau_elbow * TORQUE_TO_ACTIVATION, 0.0, 1.0)
186         data.ctrl[MUSCLE_IDS['EE']] = np.clip(data.ctrl[MUSCLE_IDS['EE']] + activation,
187         0.0, 1.0)
188         data.ctrl[MUSCLE_IDS['BE']] = max(data.ctrl[MUSCLE_IDS['BE']], activation * 0.5)
189
190     return data
191
192 # --- Main Execution Block ---
193 try:
194     # Get the angles from the user
195     TARGET_QPOS = get_user_angles()
196
197     with open(xml_path, "w") as f:
198         f.write(xml_content)
199
200     model = mujoco.MjModel.from_xml_path(xml_path)
201     data = mujoco.MjData(model)
202
203     # --- 2. Setup IDs and NEW SLOWER Gains ---
204     JOINT_IDS = [0, 1]
205     MUSCLE_IDS = {name: model.actuator(name).id for name in ['SF', 'SE', 'EF', 'EE',
206         'BF', 'BE']}
207
208     # PI-D Control Gains - REDUCED FOR SLOWER MOTION

```

```

200 KP = 120.0    # Reduced P-Gain for less aggressive initial pull
201 KI = 20.0     # Reduced I-Gain (still needed for final zero-error hold)
202 KV = 25.0     # Reduced D-Gain for less aggressive damping/stopping
203
204 # Scaling factor and Co-contraction (Keep these low for stability)
205 TORQUE_TO_ACTIVATION = 0.006
206 CO_CONTRACTION_BIAS = 0.05
207
208 sim_duration = 30.0 # Increased duration to allow the slower movement to complete
209 integral_error = np.zeros(2)
210 error_history = []
211
212 print(f"\nTarget: Shoulder {np.rad2deg(TARGET_QPOS[0]):.1f}    / Elbow
      {np.rad2deg(TARGET_QPOS[1]):.1f}    ")
213 print("Starting simulation with slower motion...")
214
215 # --- 4. Simulation Loop ---
216 with mujoco.viewer.launch_passive(model, data) as viewer:
217     start_time = data.time
218
219     while viewer.is_running() and data.time < start_time + sim_duration:
220
221         dt = model.opt.timestep
222         current_qpos = data.qpos[JOINT_IDS]
223         current_qvel = data.qvel[JOINT_IDS]
224
225         q_err = TARGET_QPOS - current_qpos
226
227         # Update Integral Term
228         integral_error += q_err * dt
229
230         # Calculate Required Torques (Joint-Space PI-D Control)
231         tau_req = KP * q_err + KI * integral_error - KV * current_qvel
232
233         # Apply Muscle Control (Synergy Mapping)
234         data = apply_muscle_control(tau_req, data, MUSCLE_IDS,
      TORQUE_TO_ACTIVATION, CO_CONTRACTION_BIAS, KP)
235
236         # Log time and the full error vector
237         error_history.append((data.time, q_err))
238
239         # Dynamics and Rendering
240         mujoco.mj_step(model, data)
241         viewer.sync()
242
243         time.sleep(dt * 0.5)
244
245     print("\n--- Simulation Complete ---")
246
247 # --- 5. Plot Dual Error vs. Time ---
248
249 times = [item[0] for item in error_history]
250
251 shoulder_errors_deg = [np.abs(np.rad2deg(item[1][0])) for item in error_history]
252 elbow_errors_deg = [np.abs(np.rad2deg(item[1][1])) for item in error_history]
253
254 plt.figure(figsize=(10, 6))
255 plt.plot(times, shoulder_errors_deg, label=f'Shoulder Error (Target:
      {np.rad2deg(TARGET_QPOS[0]):.1f}    )', color='blue')

```

```

256     plt.plot(times, elbow_errors_deg, label=f'Elbow Error (Target:
      {np.rad2deg(TARGET_QPOS[1]):.1f} )', color='red', linestyle='--')
257
258     plt.axhline(y=0.5, color='gray', linestyle=':', alpha=0.7, label='0.5 Tolerance
      Zone')
259
260     plt.title('Dual Joint Tracking Error vs. Time (Slower Muscle Control)')
261     plt.xlabel('Time (seconds)')
262     plt.ylabel('Absolute Angular Error (Degrees)')
263     plt.grid(True)
264     plt.legend()
265     plt.show()
266
267
268 except Exception as e:
269     print(f"An error occurred: {e}")
270
271 finally:
272     if os.path.exists(xml_path):
273         os.remove(xml_path)

```

Listing 1: Complete Python Control and Simulation Script